# COMP5152
# Advanced Data Analytics

Group 11


| Name | |
|---|---|
| Ho Chi Chung Joe | 21010627G |
| Chau Kai Fai Allen | 20024288G |
| Wong Yuk Hong | 21005935G |
| Chiu Pit Ho | 21030816G |
| Ko Wang Hon | 21006177G |
| Wong Sze Man | 20077672G |

# Introduction

Can Computer see an images ?

Can Computer able to identify and classify image ?

How we can make computer to see our image ?

Can we train our own image dataset to let computer know our image ?

Is it possible to make computer see an image more accurate than a human ?

Which model is better for classify our image ?

# Background - Business Scenario

- Worked as a social media network company
  - Key Business: Photo and Video Sharing Services
- Our project
  - Our Objective
    - Study and develop an image classification feature
    - Using the existing system to classify sport image
  - Our system design and goals
    - When the user posted an image
    - System can automatically know the suitable sport category
    - Involve Machine Learning and Artificial Intelligence
    - Provide more customized content to our users
    - Improve the efficiency of spotting target audience
  - Our Job
    - To Build a Machine Learning Model for identify sport image

# Methodology

- Data Preprocess
  - Gaining information on the image data
  - Image Data Normalization
  - Image Transformation
- Analysis and Evaluation
  - Multi-nominal Logistic Regression
  - Build a Convolution Neural Network
  - Use Pre-trained Model
    - MobileNetV2 (Dense)
    - MobileNetV2 (Convolutional Neural Network)
    - EffiencientNet

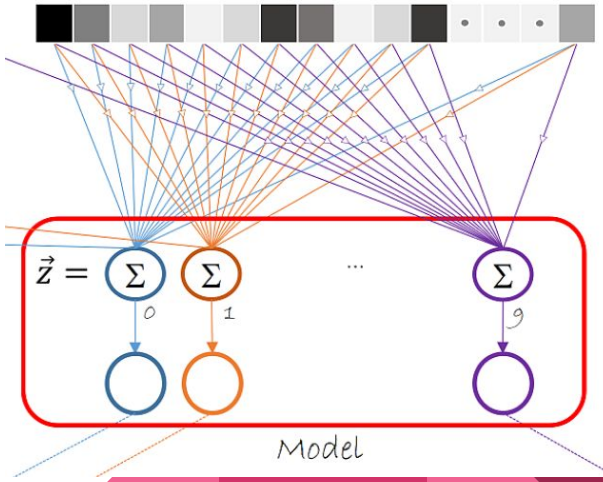
# Multinominal Logistic Regression

Input feature: 224 X 224 Pixels= 50176 features, Turn RGB (3 channels) to grayscale (1 channel)

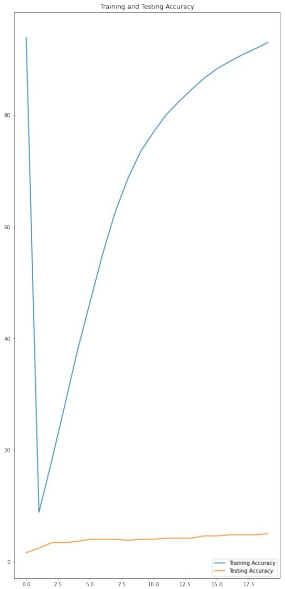
Softmax to normalized the resultant outputs (100 classes) between 0 and 1:

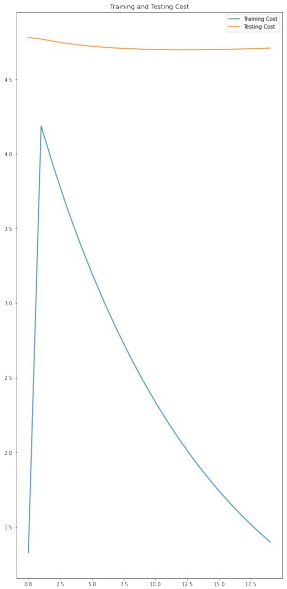
$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

The neuron which has the maximum value across 100 outputs will be encoded as 1, predicting the corresponding class.



# Multinominal Logistic Regression




Result:
Serve overfitting problem, Low accuarcy (below 10%) in testing data.

Reasons:
1. Some features contain noise, i.e. irrelvant to type of sports, such as human face colors

2. The Regression model is unable to draw more complex relationships/ decision boundaries for multi-class classification

Improvement Direction:

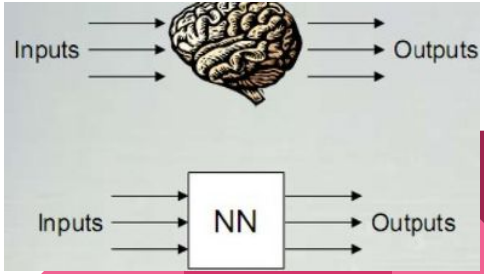
1. Use filters to remove noise.

2. Deep Neural Network to draw the decision boundaries precisely

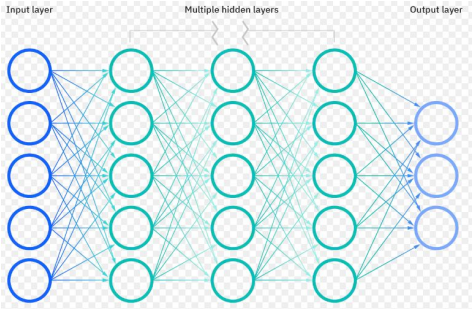
# Convolutional Neural Network

What is Artificial Neural Network ?

- A machine learning mathemtical model inspired by biological nervous system
- Consist numerous of artificial neurons and connect each together
- Consists of three layer of neurons
  - input layer, one or more hidden layers, and an output layer.
- Neurons receive number of input and perform some mathematical task and function
- Much simpler than biological neural network
- Proved Good Generic Algorithm for machine learning



# Convolutional Neural Network




What is Deep Neural Network?

- Artificial Neural Network with multiple hidden layer

What is Deep Learning

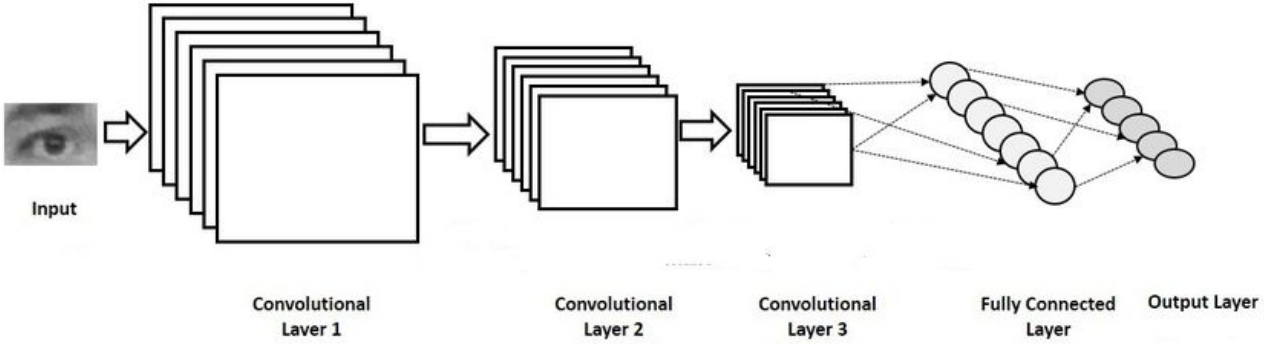
- Machine Learning Model using Deep Neural Network
- Normally Better Learning Accuracy compare with traditional artificial neural network
- Proved Great Generic Machine Learning Algorithm

# Convolutional Neural Network

An Improved Artificial Neural Network and a deep learning model

- Inspired by the organization of neuros of visual cortex in the human brain

Proved Machine Learning Algorithm for learning and processing data like images, audio and video



# Convolutional Neural Network

Input Layer

- Filters

Normalization

Convolutional Layer

Pooling Layer

Fully-connected layer

Dropout layer

# Convolutional Neural Network

Our Training Model

- Based on Tensorflow and Keras Library
- Read the target dataset training data and test data
- Set Batch Size to 32
- Normalization and Rescaling

# Convolutional Neural Network

Our Training Model

- Four Convolutional Hidden Layer with different filter
- Using LeakyReLU Activation Function
- Max Pooling Layer
- Fatten Layer
- Use Adam as Optimizers
- Padding : Valid
- 10 epoch

# Convolutional Neural Network

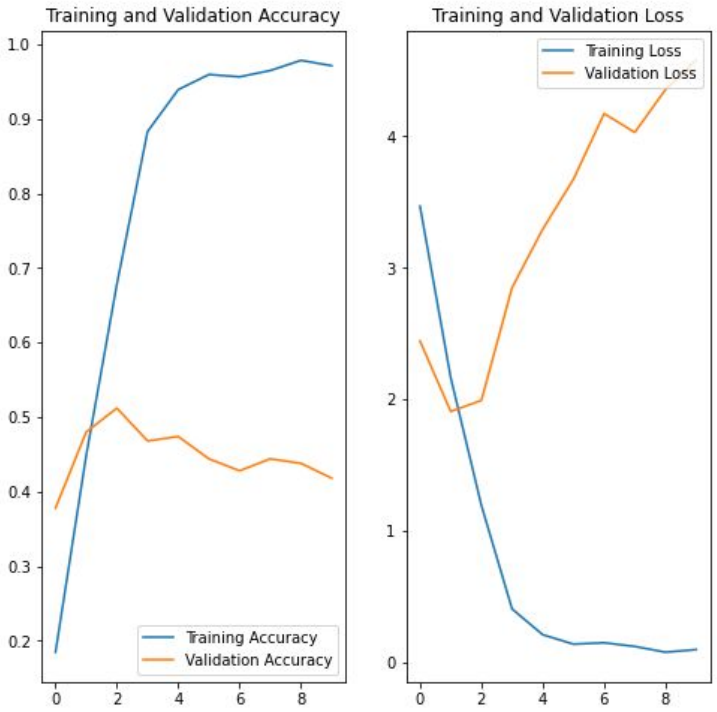
Summary of our training model

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 rescaling_1 (Rescaling)     (None, 224, 224, 3)       0

 conv2d (Conv2D)             (None, 222, 222, 16)      448

 max_pooling2d (MaxPooling2D  (None, 111, 111, 16)     0
 )

 conv2d_1 (Conv2D)           (None, 109, 109, 32)      4640

 max_pooling2d_1 (MaxPooling  (None, 54, 54, 32)       0
 2D)

 conv2d_2 (Conv2D)           (None, 52, 52, 64)        18496

 max_pooling2d_2 (MaxPooling  (None, 26, 26, 64)       0
 2D)

 conv2d_3 (Conv2D)           (None, 24, 24, 128)       73856

 max_pooling2d_3 (MaxPooling  (None, 12, 12, 128)      0
 2D)

 flatten (Flatten)           (None, 18432)             0

 dense (Dense)               (None, 192)               3539136

 dense_1 (Dense)             (None, 100)               19300

=================================================================
Total params: 3,655,876
Trainable params: 3,655,876
Non-trainable params: 0
_________________________________________________________________
```

# Convolutional Neural Network

Training Accuracy

- High Training Accuracy
- Validation Accuracy remains low



# Convolutional Neural Network

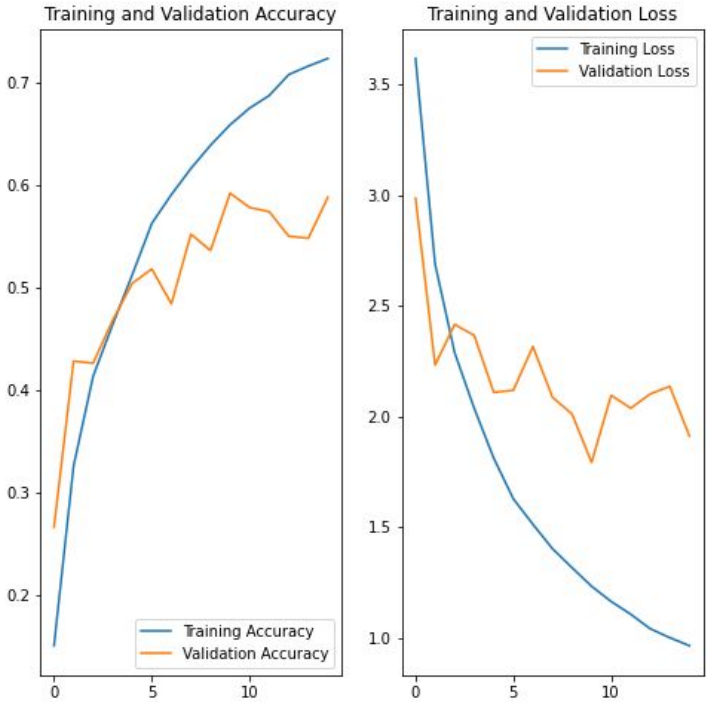
## Improvement of our model

- Data Augmentation
- Padding: Same
- Increase epoch from 10 to 15

# Convolutional Neural Network

## Results

- Lower Training Accuracy
- But Higher Validation Accuracy
- Successfully improved the model overfit problem



# MobilenNetV2(introducetion)

- Pre-trained model for classify images or detect objects
- Depth-wise separable convolution
  - Depth-wise convolution
  - Point-wise convolution
- MobilenetV2
  - Improved the performance of V1 by two new features:
    - linear bottlenecks
    - shortcut connections

Advantages:

1. fast
2. light weight

# MobileNetV2(training procedure)

1. Load training and validation dataset
2. Set batch size to 32
3. Auto tune training and validation dataset
4. Augmentation with horizontal flip
5. Add dense output layer 100 neunons after MobileNetv2 (softmax)
6. Train the model for 10 epochs

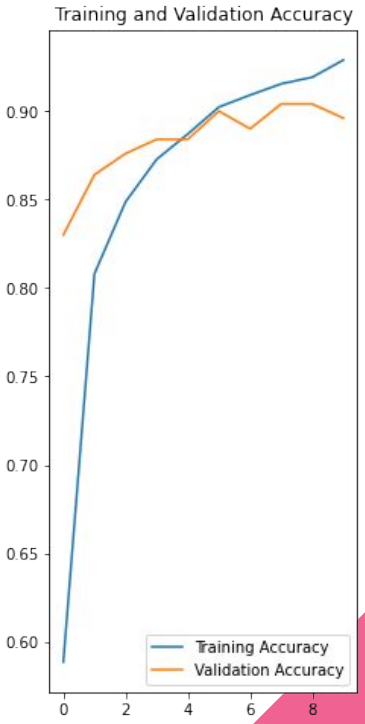
| Layer (type) | Output Shape | Param # |
|---|---|---|
| input_6 (InputLayer) | [(None, 224, 224, 3)] | 0 |
| sequential (Sequential) | (None, 224, 224, 3) | 0 |
| rescaling_2 (Rescaling) | (None, 224, 224, 3) | 0 |
| mobilenetv2_1.00_224 (Functional) | (None, 7, 7, 1280) | 2257984 |
| global_average_pooling2d_1 (GlobalAveragePooling2D) | (None, 1280) | 0 |
| dense_2 (Dense) | (None, 100) | 128100 |
| tf.nn.softmax_2 (TFOpLambda) | (None, 100) | 0 |

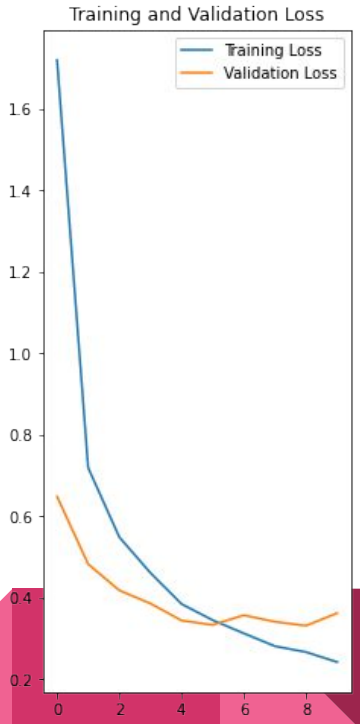
Total params: 2,386,084
Trainable params: 128,100
Non-trainable params: 2,257,984

# MobileNetV2(result)

- Highest accuray: 0.9040
- Overfit at the 9th epoch

```
Epoch 1/10
425/425 [==============================] - 318s 742ms/step - loss: 1.7204 - accuracy: 0.5885 - val_loss: 0.6486 - val_accuracy: 0.8300
Epoch 2/10
425/425 [==============================] - 321s 755ms/step - loss: 0.7190 - accuracy: 0.8081 - val_loss: 0.4825 - val_accuracy: 0.8640
Epoch 3/10
425/425 [==============================] - 312s 734ms/step - loss: 0.5489 - accuracy: 0.8487 - val_loss: 0.4181 - val_accuracy: 0.8760
Epoch 4/10
425/425 [==============================] - 304s 715ms/step - loss: 0.4610 - accuracy: 0.8728 - val_loss: 0.3858 - val_accuracy: 0.8840
Epoch 5/10
425/425 [==============================] - 304s 714ms/step - loss: 0.3847 - accuracy: 0.8870 - val_loss: 0.3439 - val_accuracy: 0.8840
Epoch 6/10
425/425 [==============================] - 303s 712ms/step - loss: 0.3449 - accuracy: 0.9023 - val_loss: 0.3332 - val_accuracy: 0.9000
Epoch 7/10
425/425 [==============================] - 303s 713ms/step - loss: 0.3126 - accuracy: 0.9091 - val_loss: 0.3570 - val_accuracy: 0.8900
Epoch 8/10
425/425 [==============================] - 303s 713ms/step - loss: 0.2810 - accuracy: 0.9154 - val_loss: 0.3410 - val_accuracy: 0.9040
Epoch 9/10
425/425 [==============================] - 303s 714ms/step - loss: 0.2665 - accuracy: 0.9192 - val_loss: 0.3312 - val_accuracy: 0.9040
Epoch 10/10
425/425 [==============================] - 303s 714ms/step - loss: 0.2421 - accuracy: 0.9289 - val_loss: 0.3621 - val_accuracy: 0.8960
```



# MobileNetV2(optimization)

Method 1: Add 3 CNN layers between mobilenetv2 and output layer

We expect it can improve the performance as the CNN can capture the spatial feature from the output of the mobilenetv2.

Method2: Add 2 more dense layer with dropout layer  between the mobilenetv2 and output layer

We expect 2 more dense layers can learn more features to classify the images and the dropout layer can reduce the overfit.

# MobileNetV2 optimization(Add 3 CNN layers)

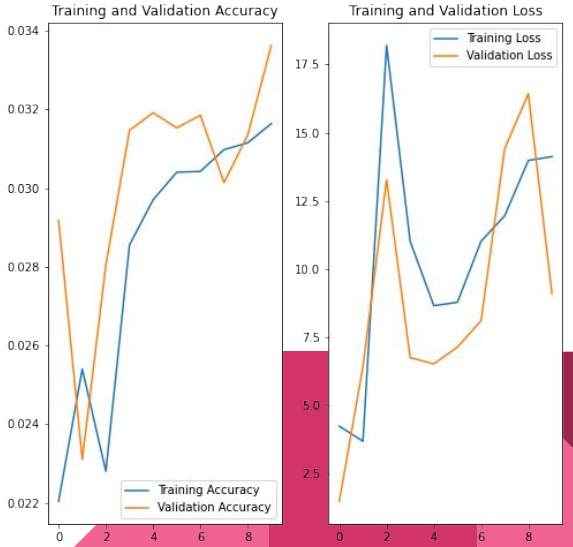
The CNN layers output shape:
(5,5,512),(3,3,256),(1,1,128)

Result: Around 0.033 accuracy (no improvement)

Potential reason of the bad accuracy :

1. (1,1,128) CNN layers lost the features
2. Model is not optimized as the validation accuracy still higher than the train accuracy

```
mobilenetv2_1.00_224 (Funct  (None, 7, 7, 1280)      2257984
ional)

conv2d (Conv2D)              (None, 5, 5, 512)       5898752

conv2d_1 (Conv2D)            (None, 3, 3, 256)       1179904

conv2d_2 (Conv2D)            (None, 1, 1, 128)       295040
```

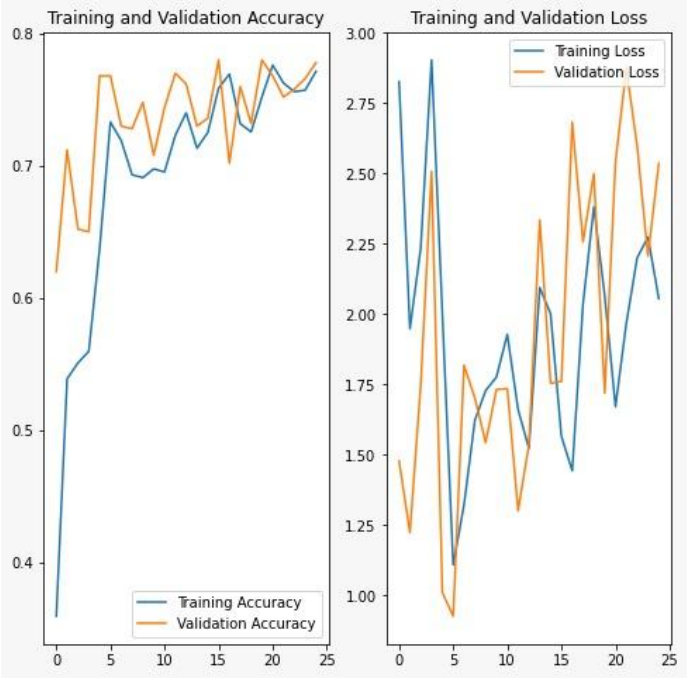


# MobileNetV2 optimization(Add 3 CNN layers-improvement)

Change CNN layers output shape:
(7,7,512),(7,7,256),(7,7,128)

Result: Around 0.78 accuracy (great improvement)

```
mobilenetv2_1.00_224 (Funct  (None, 7, 7, 1280)       2257984
ional)

conv2d (Conv2D)              (None, 7, 7, 512)        2621952

conv2d_1 (Conv2D)            (None, 7, 7, 256)        524544

conv2d_2 (Conv2D)            (None, 7, 7, 128)        131200
```

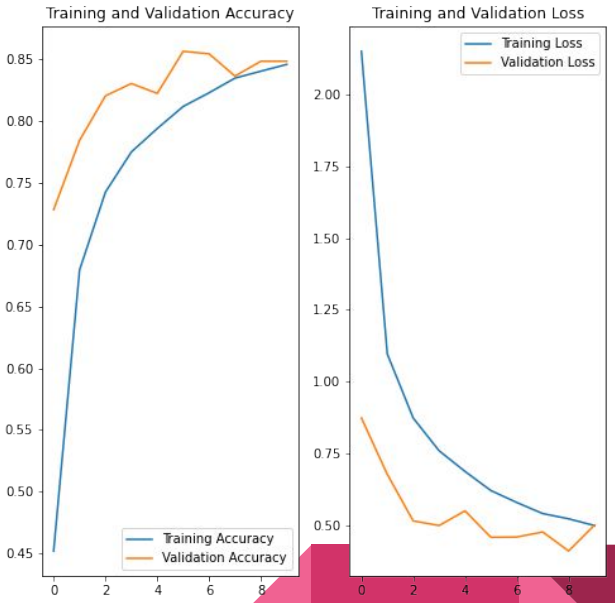


# MobileNetV2 optimization(Add 2 more dense layer)

Dense layers output shape:
(7,7,512),(7,7,256),(7,7,128)

Result: Around 0.85 accuracy

| | | |
|---|---|---|
| dense (Dense) | (None, 256) | 327936 |
| tf.nn.relu (TFOpLambda) | (None, 256) | 0 |
| dropout (Dropout) | (None, 256) | 0 |
| dense_1 (Dense) | (None, 256) | 65792 |
| tf.nn.relu_1 (TFOpLambda) | (None, 256) | 0 |
| dropout_1 (Dropout) | (None, 256) | 0 |



# MobileNetV2 optimization

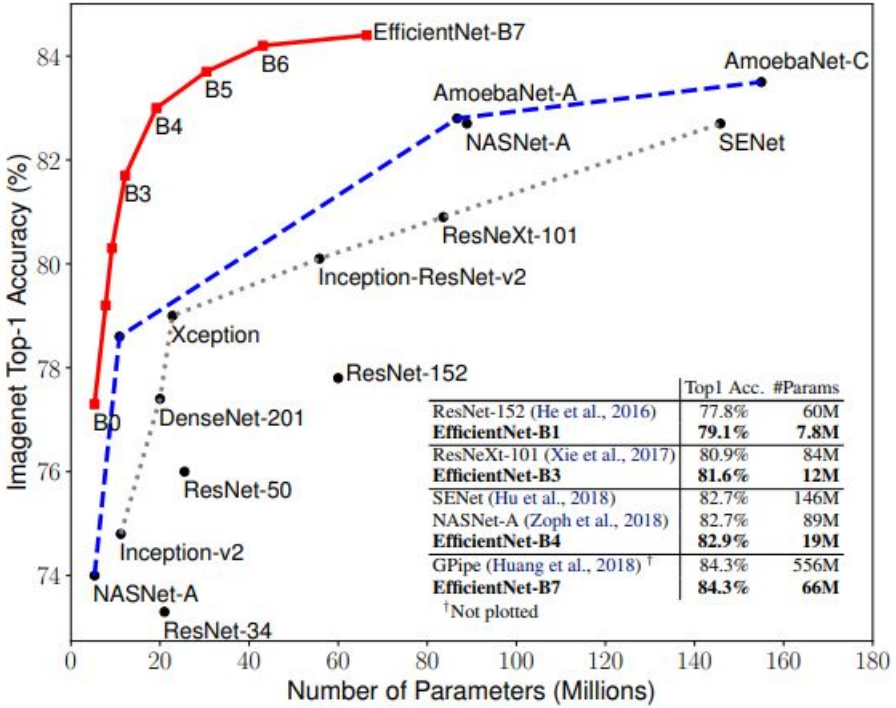
| | Direct to output layer | 3CNN | 2 Dense layer |
|---|---|---|---|
| Accuracy | 0.9040 | 0.78 | 0.85 |

Failed to improve the model

# EfficientNetB3

~ EfficientNet is a set of lightweight network based on Neural Architecture Search (NAS) released by Google in 2019

~ NAS can automatically find the best neural architecture according to our needs, and our search goal may be based on performance or hardware resource constraints.




| | Top1 Acc. | #Params |
|---|---|---|
| ResNet-152 (He et al., 2016) | 77.8% | 60M |
| **EfficientNet-B1** | **79.1%** | **7.8M** |
| ResNeXt-101 (Xie et al., 2017) | 80.9% | 84M |
| **EfficientNet-B3** | **81.6%** | **12M** |
| SENet (Hu et al., 2018) | 82.7% | 146M |
| NASNet-A (Zoph et al., 2018) | 82.7% | 89M |
| **EfficientNet-B4** | **82.9%** | **19M** |
| GPipe (Huang et al., 2018) † | 84.3% | 556M |
| **EfficientNet-B7** | **84.3%** | **66M** |

†Not plotted

# EfficientNetB3

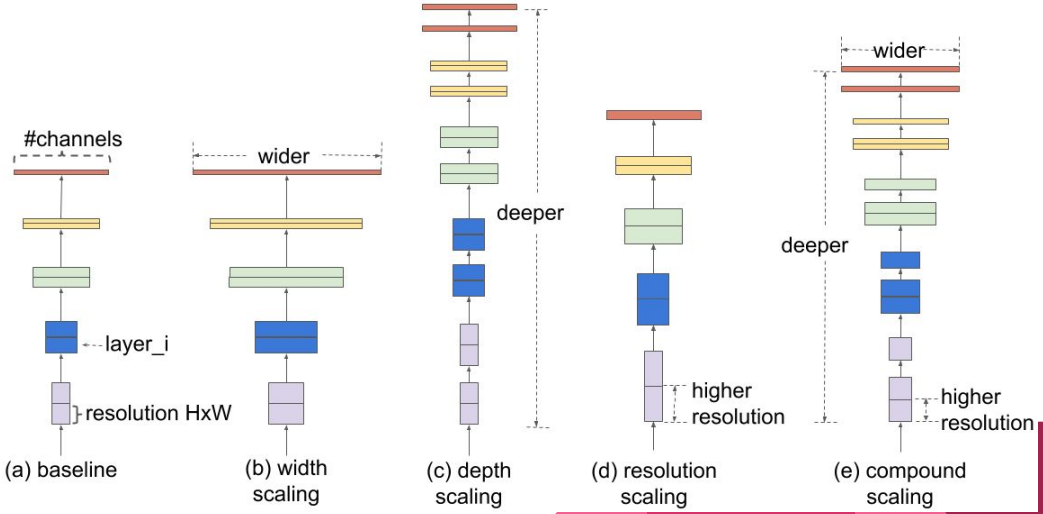
~ Traditional methods to improve the performance of neural networks are mainly based on the width of the network, the depth of the network, the and the resolution of the input image

***"EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks"***

Author found through experiments that balancing these three dimensions is crucial to the improvement of accuracy and efficiency.

Author summed up how to balance the three dimensions through a series of experiments.

Author built a **total of 7 networks B1-B7** in the EfficientNet series on the basis of EfficientNet_B0, and under the same FLOPS and parameters, the accuracy reached state-of- the-art effect.



# EfficientNetB3 Training Procedure

Referenced *"Sports using CSV File F1 score=96.7%"*
https://www.kaggle.com/code/gpiosenka/sports-using-csv-file-f1-score-96-7

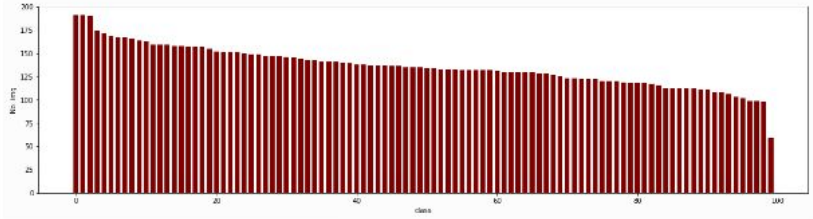
Number of training images per sport is not balanced.

> Trim the training data between **150 and 50 samples**.

> Used the ***imagedatagenerator*** in the tensorflow to do the **image augmentation and** make all sports with **170 numbers of images** for training.

> Load the EfficientNetB3 by tf.keras.applications.efficientnet.EfficientNetB3,
We also added 2 dense layers for the output for our task.

> Used Google colab for model training.(Inorder to use GPU.)

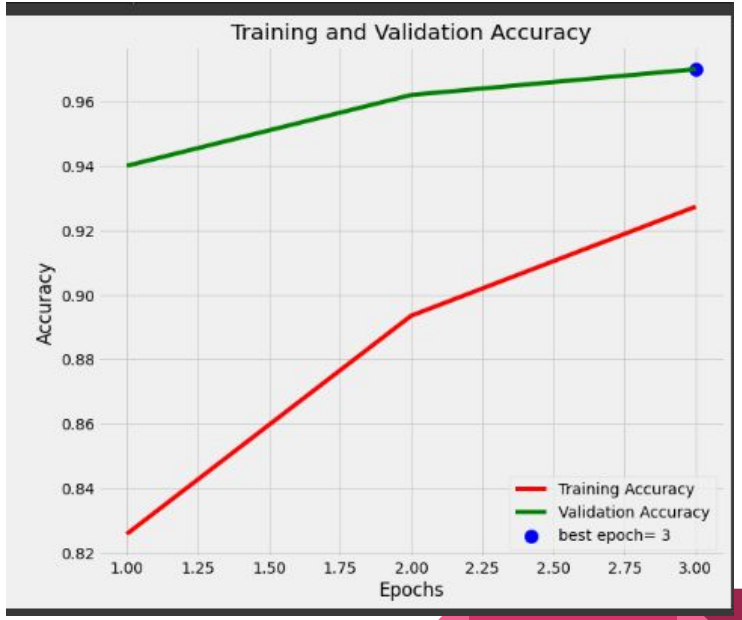


football has the most images = 191
sky surfing has the least images = 59

```
gen=ImageDataGenerator(
horizontal_flip=True,     # Do random horizontal flips
rotation_range=20,        # The angle at which the picture is rotated randomly
width_shift_range=.2,     # The magnitude of the random horizontal offset of the picture
height_shift_range=.2,    # The magnitude of the random vertical offset of the picture
zoom_range=.2             # Amount of random scaling
)
```

# EfficientNetB3 Training Procedure

Referenced *"Sports using CSV File F1 score=96.7%"*
https://www.kaggle.com/code/gpiosenka/sports-using-csv-file-f1-score-96-7




**train_loss: 1.3379 / train_accuracy: 0.9273**
**val _loss: 0.9662 / val_accuracy: 0.9700**

Referenced *"Sports using CSV File F1 score=96.7%"*
https://www.kaggle.com/code/gpiosenka/sports-using-csv-file-f1-score-96-7

# EfficientNetB3 Training Procedure

## For prediction, we get accuracy with 0.9679

| | | | | |
|---|---|---|---|---|
| accuracy | | | 0.9679 | 499 |
| macro avg | 0.9743 | 0.9680 | 0.9674 | 499 |
| weighted avg | 0.9742 | 0.9679 | 0.9674 | 499 |

```
[ ] test_valid_img_path = pathlib.Path("/content/drive/MyDrive/Colab Notebooks/comp5152/test/cricket/1.jpg")
    img_size=(224,224)
    img = tf.keras.utils.load_img(
            test_valid_img_path, target_size=(224,224)
    )
    img_array = tf.keras.utils.img_to_array(img)
    img_array = tf.expand_dims(img_array, 0) # Create a batch

    preds = model.predict(img_array)
    pred_index = np.argmax(preds)
    list(train_gen.class_indices.keys())[pred_index]

    'cricket'
```

```
10/10 [==============================] - 26s 3s/step
there were 16 in 499 tests for an accuracy of  96.79
Classification Report:
```

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| air hockey | 1.0000 | 1.0000 | 1.0000 | 4 |
| ampute football | 1.0000 | 1.0000 | 1.0000 | 5 |
| archery | 1.0000 | 1.0000 | 1.0000 | 5 |
| arm wrestling | 1.0000 | 1.0000 | 1.0000 | 5 |
| axe throwing | 1.0000 | 1.0000 | 1.0000 | 5 |
| balance beam | 1.0000 | 1.0000 | 1.0000 | 5 |
| barell racing | 1.0000 | 1.0000 | 1.0000 | 5 |
| baseball | 1.0000 | 0.6000 | 0.7500 | 5 |
| basketball | 0.8333 | 1.0000 | 0.9091 | 5 |
| baton twirling | 1.0000 | 1.0000 | 1.0000 | 5 |
| bike polo | 1.0000 | 1.0000 | 1.0000 | 5 |
| billiards | 1.0000 | 1.0000 | 1.0000 | 5 |
| bmx | 1.0000 | 0.6000 | 0.7500 | 5 |
| bobsled | 1.0000 | 1.0000 | 1.0000 | 5 |
| bowling | 1.0000 | 1.0000 | 1.0000 | 5 |
| boxing | 1.0000 | 1.0000 | 1.0000 | 5 |
| bull riding | 1.0000 | 1.0000 | 1.0000 | 5 |
| bungee jumping | 1.0000 | 0.8000 | 0.8889 | 5 |
| canoe slamon | 1.0000 | 1.0000 | 1.0000 | 5 |
| cheerleading | 1.0000 | 0.8000 | 0.8889 | 5 |
| chuckwagon racing | 1.0000 | 1.0000 | 1.0000 | 5 |
| cricket | 0.7143 | 1.0000 | 0.8333 | 5 |
| croquet | 1.0000 | 1.0000 | 1.0000 | 5 |
| curling | 1.0000 | 1.0000 | 1.0000 | 5 |
| disc golf | 1.0000 | 1.0000 | 1.0000 | 5 |
| fencing | 1.0000 | 0.8000 | 0.8889 | 5 |

# Evaluation

| | Multinomial Logistic Regression | Deep Neural Network | | | |
|---|---|---|---|---|---|
| | | CNN | mobilenetv2 (dense) | mobilenetv2 (CNN) | EffiencientNet |
| Accuracy | 0.05 | 0.566 | 0.904 | 0.78 | 0.968 |

Best

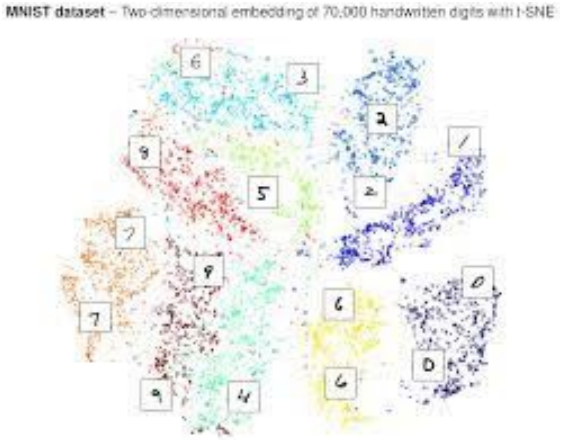
# Model Performance Visualization

An in-depth visualize the relationship between each class/image.

| Predicted Class \ True Class | Positive | Negative |
|---|---|---|
| Positive | TP | FP |
| Negative | FN | TN |

## Confusion Matrix

**A summary of prediction results**




## t-SNE

**Visualize the model how to classify images**

# Effiencient Net

Highest accuracy model we trained

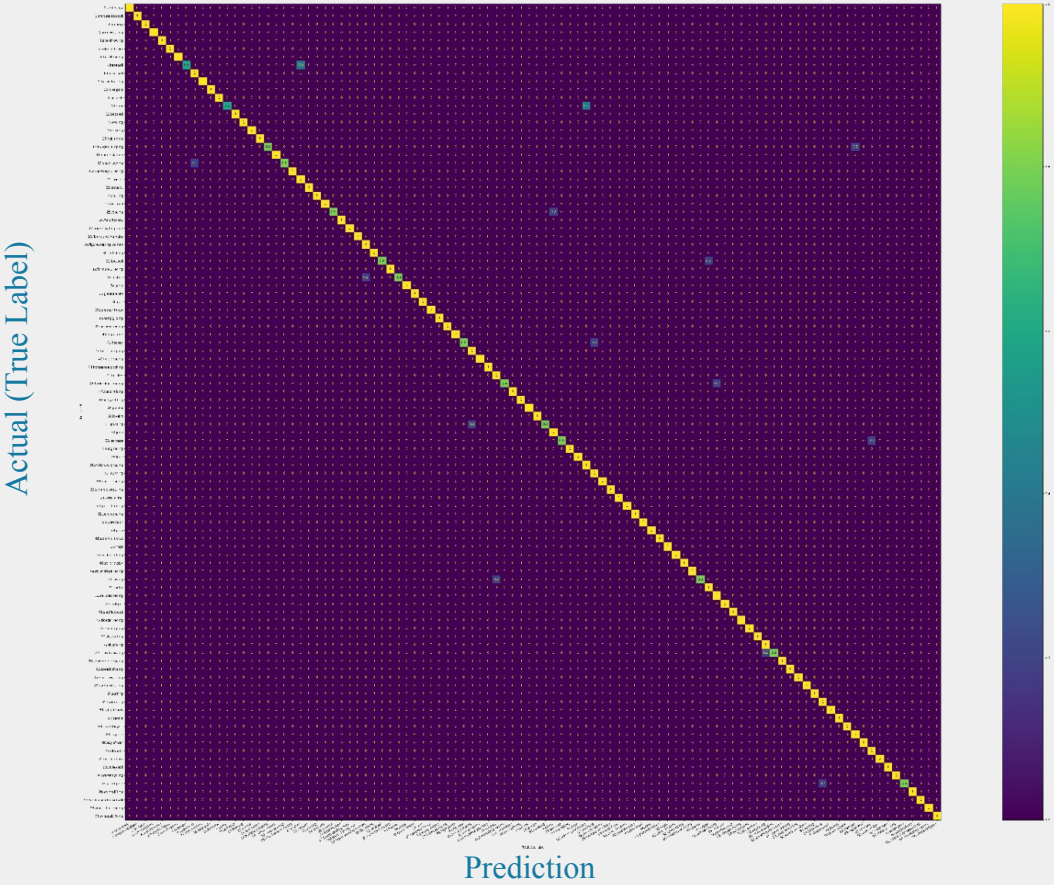
**Prediction with Test Dataset**

Test Dataset contains:

- 100 classes
- 5 images/ class
- Total 500 images

| | |
|---|---|
| 1 | 100% - 5 out of 5 images |
| 0.8 | 80% - 4 out of 5 images |
| 0.6 | 60% - 3 out of 5 images |
| 0.4 | 40% - 2 out of 5 images |
| 0.2 | 20% - 1 out of 5 images |
| 0 | 0% - 0 out of 5 images |

## Confusion Matrix



# Effiencient Net

Highest accuracy model we trained

Most of the predictions are **correct**

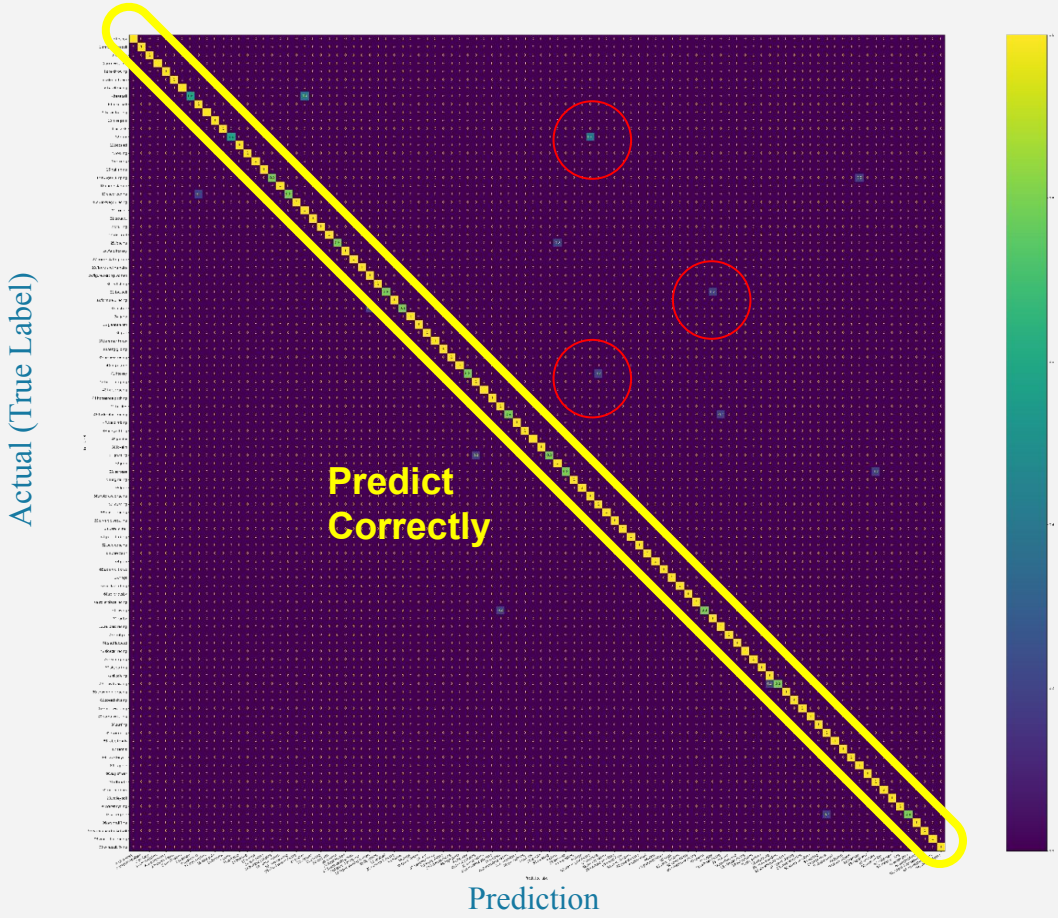
Only few of classes **predict wrongly**

E.g.

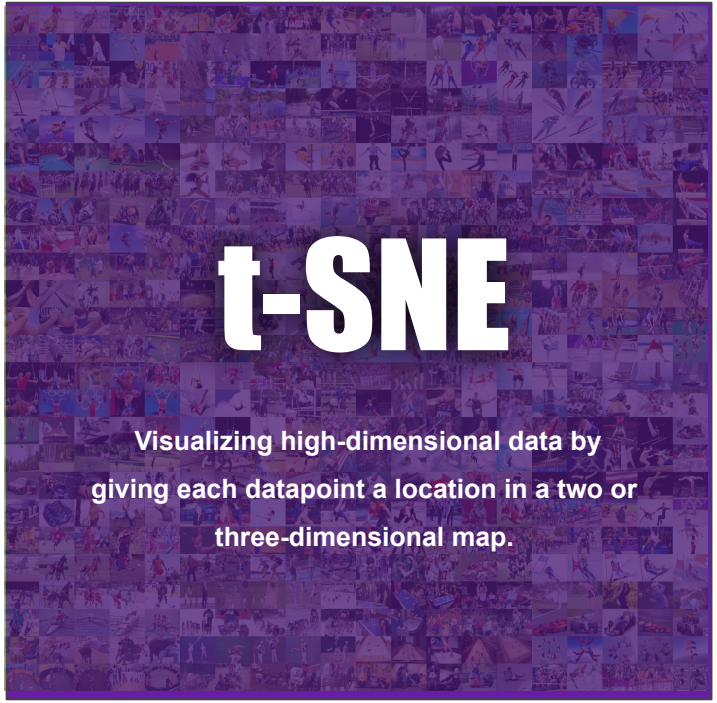
| | | Prediction | |
|---|---|---|---|
| | | baseball | cricket |
| True Label | baseball | 0.6 | 0.4 |
| | cricket | 0 | 1 |

| | | Prediction | |
|---|---|---|---|
| | | bmx | motorcycle racing |
| True Label | bmx | 0.6 | 0.4 |
| | motorcycle racing | 0 | 1 |

## Confusion Matrix



To insight what happen in the model…

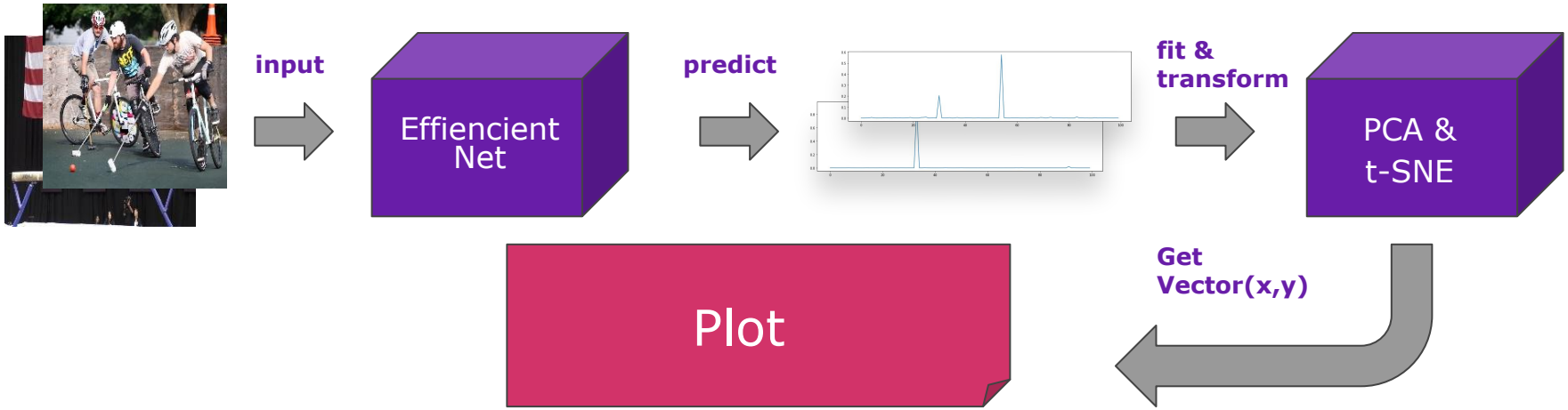



Visualize the feature vector learned by the model from training

# t-SNE viusalize Effiencient Net model how to classify images

**Method**

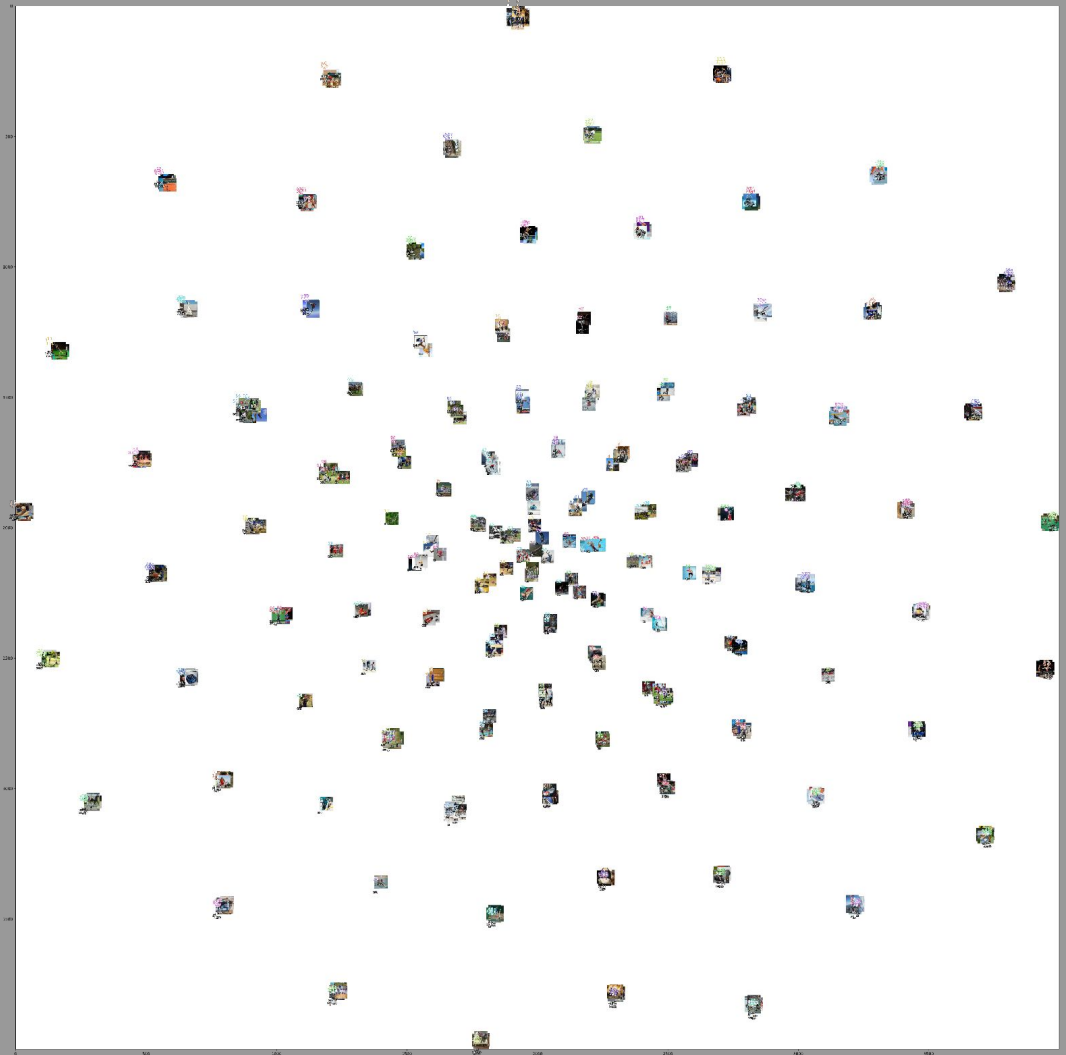
1. Predict image by the trained model - Effiencient Net
2. Collect the output of the last layer of the model, can be seen as feature vector
3. fit the output into the PCA and t-SNE algorithm , will get 2 demsional vectors
4. Plot the graph



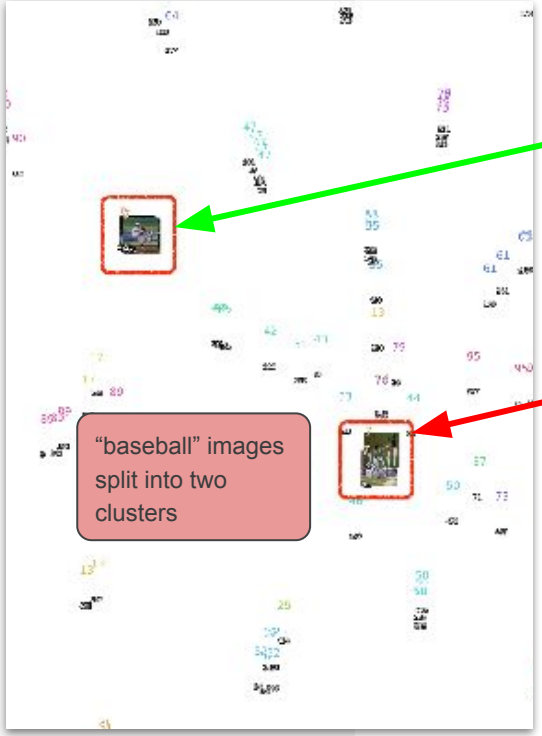
# T-SNE Result

- Each cluster is clearly separate
- Images in the same cluster are very close to each other.

**Model is quite confident to classify the images.**



# Insights from Wrong Prediction




T-SNE plot

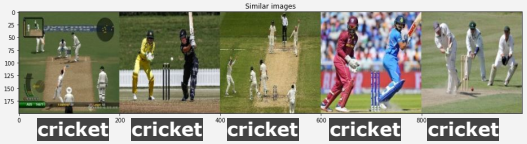
**ALL 5 images labeled as "baseball"**

Predict sucessfully!

Prediction failed!

```
query_class 74 : baseball
closest image 102 : cricket
closest image 10 : cricket
closest image 223 : cricket
closest image 465 : cricket
closest image 432 : cricket
```

Similar images with close features



Prediction failed!

```
query_class 130 : baseball
closest image 74 : baseball
closest image 10 : cricket
closest image 102 : cricket
closest image 223 : cricket
closest image 465 : cricket
```

Similar images with close features

For improving the model, we might consider to input more images with similar feature of incorrect prediction images to the training image set

# Q & A